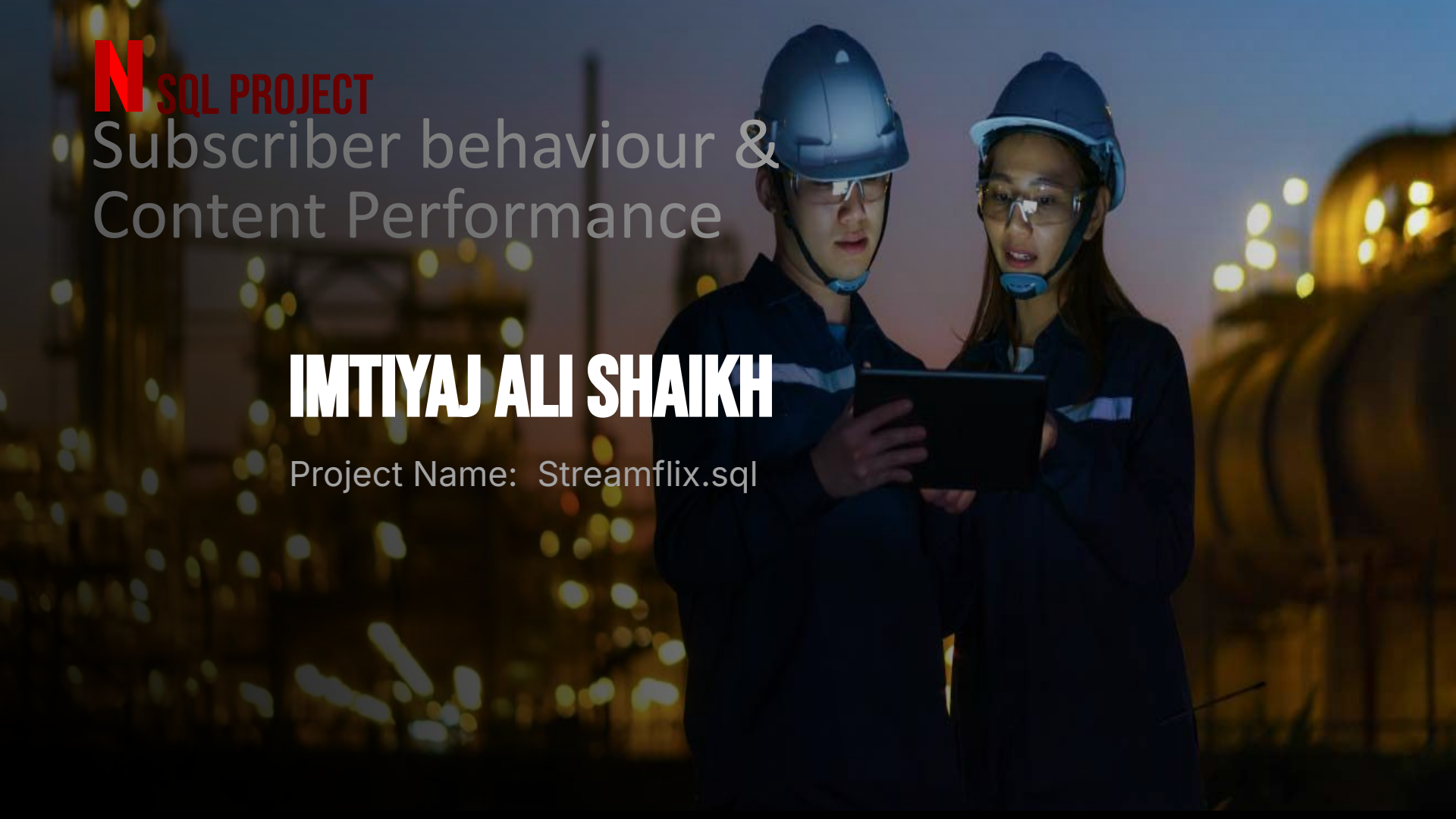


N SQL PROJECT

Subscriber behaviour & Content Performance

IMTIYAJ ALI SHAIKH

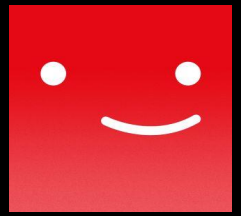
Project Name: Streamflix.sql

STREAMFLIX

Subscriber Behavior & Content Performance Analysis

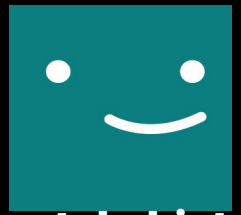
Database Project1

Show tables:
select * from users;
select * from content;
select * from watch_history;
select * from reviews;



users

content



watch_histor
y

reviews

Result Grid | Filter Rows: Search | Edit:

user_id	user_name	email	subscription_tier	join_date
1	Alice	alice@email.com	Premium	2023-01-15
2	Bob	bob@email.com	Basic	2023-03-20
3	Charlie	charlie@email.com	Standard	2023-05-10
4	Diana	diana@email.com	Premium	2023-08-05
5	Evan	evan@email.com	Basic	2024-01-02
NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows: Search | Edit:

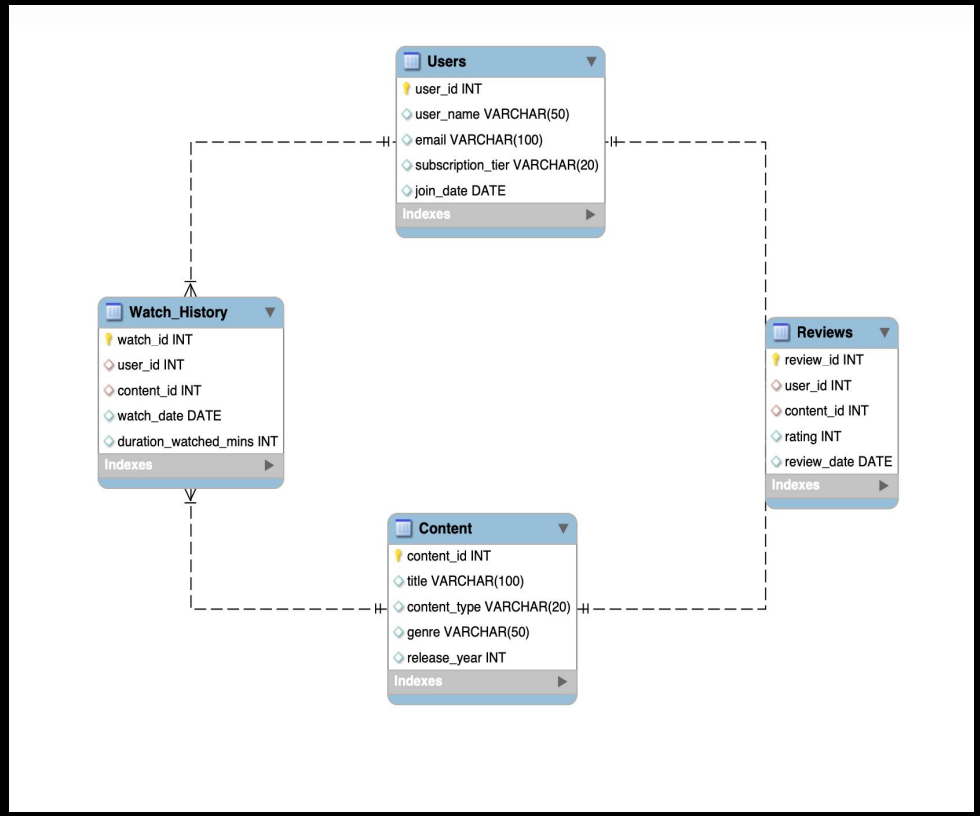
content_id	title	content_type	genre	release_year
101	Galactic Wars	Movie	Sci-Fi	2022
102	The Detective	Show	Mystery	2021
103	Laugh Out Loud	Movie	Comedy	2023
104	Deep Ocean	Documentary	Nature	2020
105	Mystery House	Movie	Mystery	2023
NULL	NULL	NULL	NULL	NULL

watch_id	user_id	content_id	watch_date	duration_watched_mins
1001	1	101	2024-02-01	120
1002	1	102	2024-02-02	45
1003	2	103	2024-02-01	90
1004	3	101	2024-02-03	120
1005	1	102	2024-02-04	45
1006	4	105	2024-02-05	110
1007	2	101	2024-02-05	30
NULL	NULL	NULL	NULL	NULL

review_id	user_id	content_id	rating	review_date
501	1	101	5	2024-02-02
502	2	103	4	2024-02-02
503	3	101	3	2024-02-04
504	2	101	1	2024-02-06
NULL	NULL	NULL	NULL	NULL

SQL PROJECT

Streamflix EER diagram



THE PLOT

98% Match 2026 SQL-16+ 4 Tables

This project explores a relational database named **Project1** , analyzing user activity and content performance across four interconnected tables. We dive deep into subscriber behavior and content engagement metrics.

USERS

Profile data, subscription tiers (Basic, Standard, Premium).

CONTENT

Catalog of movies and shows with genre and release metadata.

WATCH HISTORY

Transactional records of user engagement and duration.

REVIEWS

User-generated feedback with ratings on a 1–5 scale.

EPISODE 1

THE POWER OF JOINS

THE INSIGHT

By merging user profiles with watch history, we identify which content drives the most engagement across different subscription tiers.

BUSINESS VALUE

Identifying inactive reviewers allows for targeted re-engagement campaigns to boost platform interaction.

JOIN S

1. Inner Join: Write a query to display the user_name, title of the content they watched, and the watch_date.

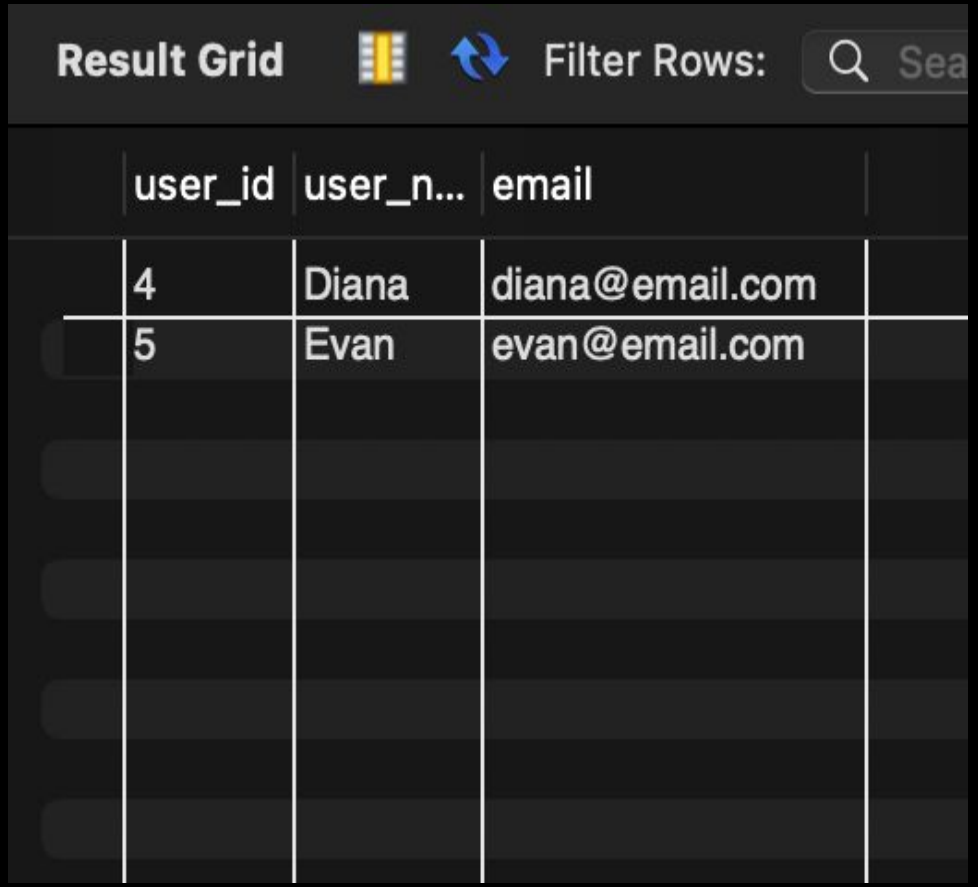
```
select  u.user_name as user_name,
                c.title as Title_of_Content,
                        w.watch_date
from users u
Inner join watch_history w
on u.user_id = w.user_id
Inner join content c on w.content_id =
c.content_id
order by user_name;
```

user_n...	Title_of_Conte...	watch_date
Alice	Galactic Wars	2024-02-01
Alice	The Detective	2024-02-02
Alice	The Detective	2024-02-04
Bob	Laugh Out Loud	2024-02-01
Bob	Galactic Wars	2024-02-05
Charlie	Galactic Wars	2024-02-03
Diana	Mystery House	2024-02-05

JOINS

2. Left Join: Find all Users who have never written a review. Display their user_name and email.

```
select u.user_id, u.user_name as
        user_name,
                u.email as email
from users u
Left join reviews r on u.user_id = r.user_id
where r.user_id is null
order by u.user_id
;
```



user_id	user_n...	email
4	Diana	diana@email.com
5	Evan	evan@email.com

JOIN S

3. Multiple Joins: Get a list of user_names, the title of the content they reviewed, and the rating they gave it.

```
select      u.user_name,
            c.title, r.rating
from users u
join watch_history w
on u.user_id = w.user_id
join content c
on w.content_id = c.content_id
join reviews r
on c.content_id = r.content_id
order by u.user_name
;
```

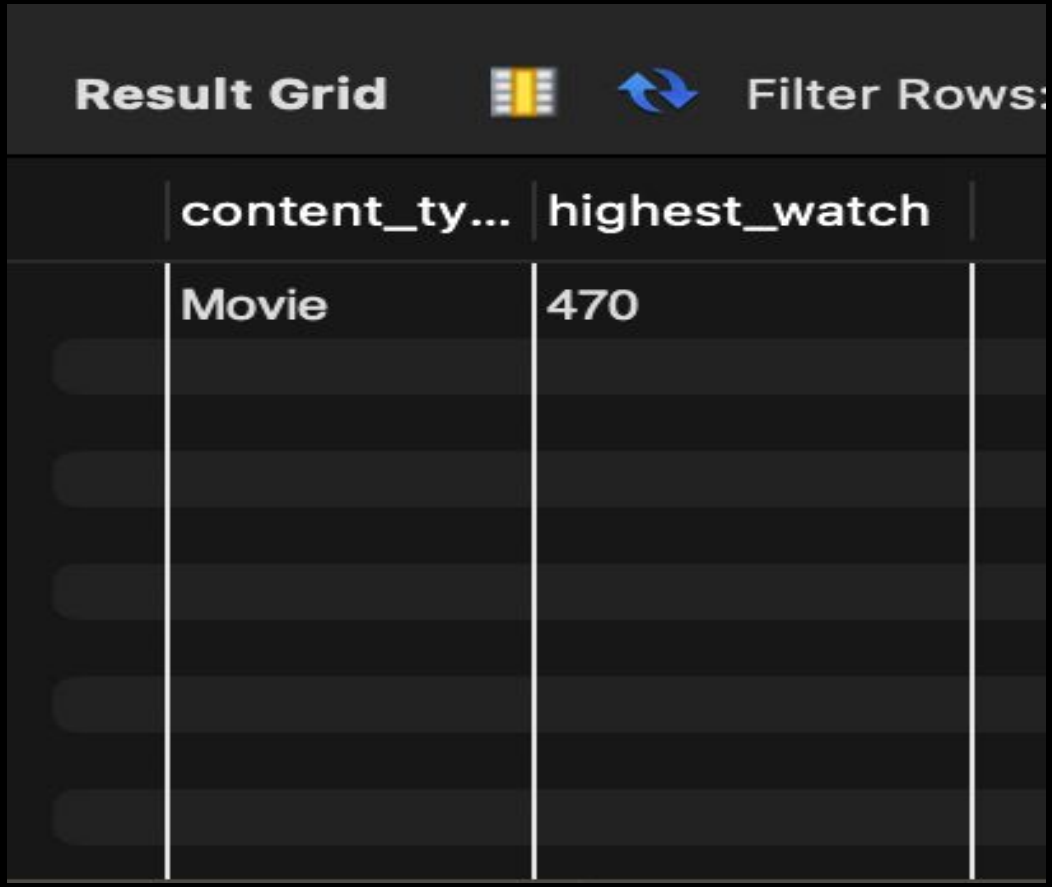
Result Grid Filter Rows:

user_n...	title	rating
Alice	Galactic Wars	5
Alice	Galactic Wars	3
Alice	Galactic Wars	1
Bob	Galactic Wars	5
Bob	Laugh Out Loud	4
Bob	Galactic Wars	3
Bob	Galactic Wars	1
Charlie	Galactic Wars	5
Charlie	Galactic Wars	3
Charlie	Galactic Wars	1

JOIN S

4. Aggregation + Join: Which genre has the highest total number of watch minutes across all users?

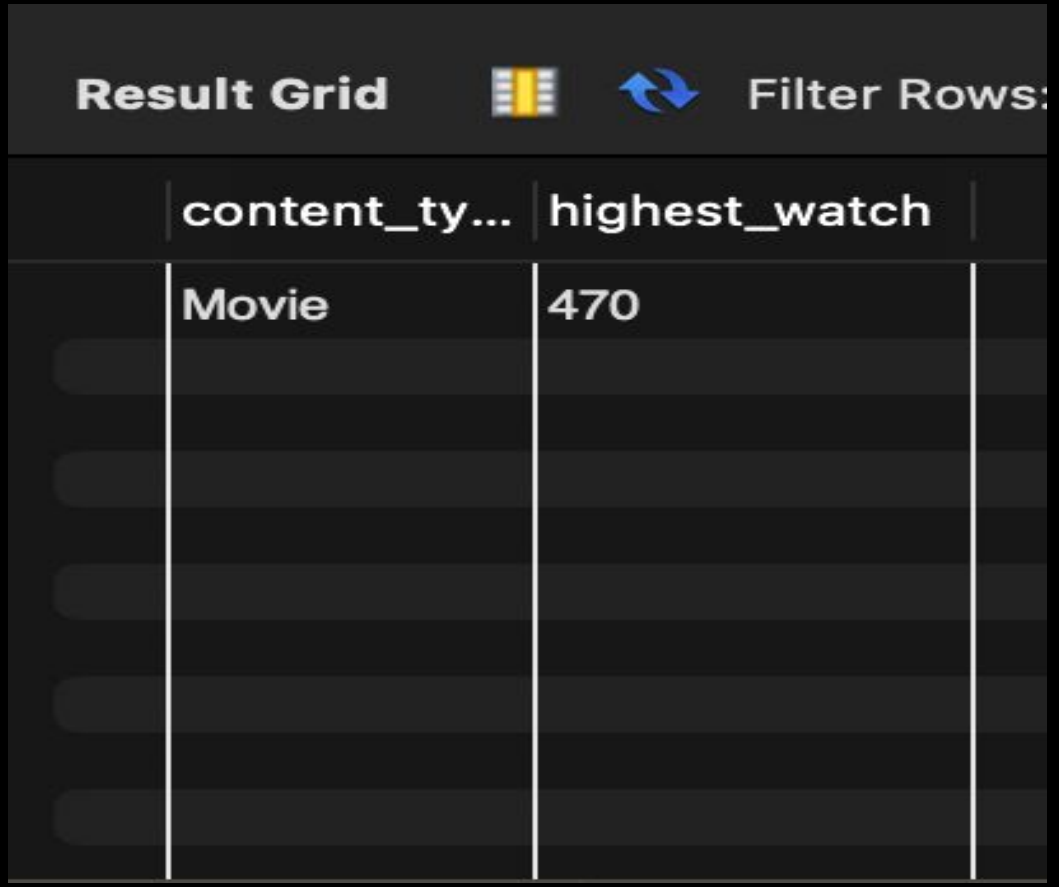
```
select c.content_type,
sum(w.duration_watched_mins) as
highest_watch
from content c
join watch_history w on c.content_id =
w.content_id
group by c.content_type
order by highest_watch desc limit 1
;
```



JOIN
S

5. Filtering with Joins: List the titles of all content watched by users who are on the 'Premium' subscription_tier.

```
select u.user_name, u. subscription_tier,
c.title
from users u
join watch_history w on u.user_id =
w.user_id
join content c on w.content_id =
c.content_id
where u.subscription_tier = 'Premium'
order by u.user_name
;
```



EPISODE 2

WINDOW FUNCTIONS

DYNAMIC TREND ANALYSIS

Moving beyond static aggregations, window functions allow us to analyze user behavior in sequence, identifying "binge" patterns and relative content performance.

BUSINESS IMPACT

Enables personalized content recommendations by tracking how user interest evolves over time and identifying high-retention content through dense ranking.

Used to calculate running totals, content leaderboards, and sequential watch gaps.

Window Functions

6. Running Total: Calculate the running total of duration_watched_mins for User 'Alice' ordered by her watch_date.

```
select      u.user_name,
            w.watch_date,
        w.duration_watched_mins,
        sum(w.duration_watched_mins)
over(partition by u.user_id order by
w.watch_date) as running_total
from users u
join watch_history w
on u.user_id = w.user_id
where u.user_name = 'Alice'
order by w.watch_date;
```

Window Functions

7. Ranking: Rank the Content (title) based on their average review rating from highest to lowest. Use DENSE_RANK().

```
select c.title,
avg(r.rating),
dense_rank() over(order by avg(r.rating)
desc) as Ranking
from content c
join reviews r on c.content_id =
r.content_id
group by c.title
;
```

Window Functions

8. Partitioning: For each subscription_tier, find the user who has watched the most total minutes of content.

```
with user_watch_time as (select  u.user_id,
u.user_name, u.subscription_tier,
sum(w.duration_watched_mins) as total_Minutes
from users u
join watch_history w on u.user_id = w.user_id
group by u.user_id,
                u.user_name,
        u.subscription_tier
        )
select * from ( select *,rank() over(partition by
subscription_tier order by total_Minutes desc) as
rnk from user_watch_time) t
where rnk = 1
order by user_id;
```

Result Grid Filter Rows: Search Expo

user_id	user_n...	subscription_tier	total_Minut...	rnk
1	Alice	Premium	210	1
2	Bob	Basic	120	1
3	Charlie	Standard	120	1

Window Functions

9. Lag/Lead: For User 'Alice', write a query that shows her current watch_date and the watch_date of the previous thing she watched.

```
select  u.user_id, u.user_name, w.watch_date,
        c.title, lag(w.watch_date) over(partition by
u.user_id order by w.watch_date) as
pre_watch_date
from users u
join watch_history w
on u.user_id = w.user_id
join content c
on w.content_id = c.content_id
where u.user_name = 'Alice'
order by u.user_id, w.watch_date;
```

Result Grid | Filter Rows: Search | Exp

user_id	user_n...	watch_date	title	pre_watch_date
1	Alice	2024-02-01	Galactic Wars	NULL
1	Alice	2024-02-02	The Detective	2024-02-01
1	Alice	2024-02-04	The Detective	2024-02-02

Window Functions

10. Percentiles: Use NTILE(4) to divide all content into 4 quartiles based on their release year, from newest to oldest.

```
select
        title,
    release_year,
    ntile(4) over(order by release_year desc,
content_id) as quartile
from content
;
```

Result Grid   Filter Rows:  Sea

title	release_year	quartile
Laugh Out Loud	2023	1
Mystery House	2023	1
Galactic Wars	2022	2
The Detective	2021	3
Deep Ocean	2020	4

EPISODE 3

DEEP DIVE SUBQUERIES

03

PRECISION FILTERING

Leveraging subqueries to isolate specific content interactions without the overhead of multiple joins. This approach ensures data integrity when querying nested relationships.

SQL HIGHLIGHTS

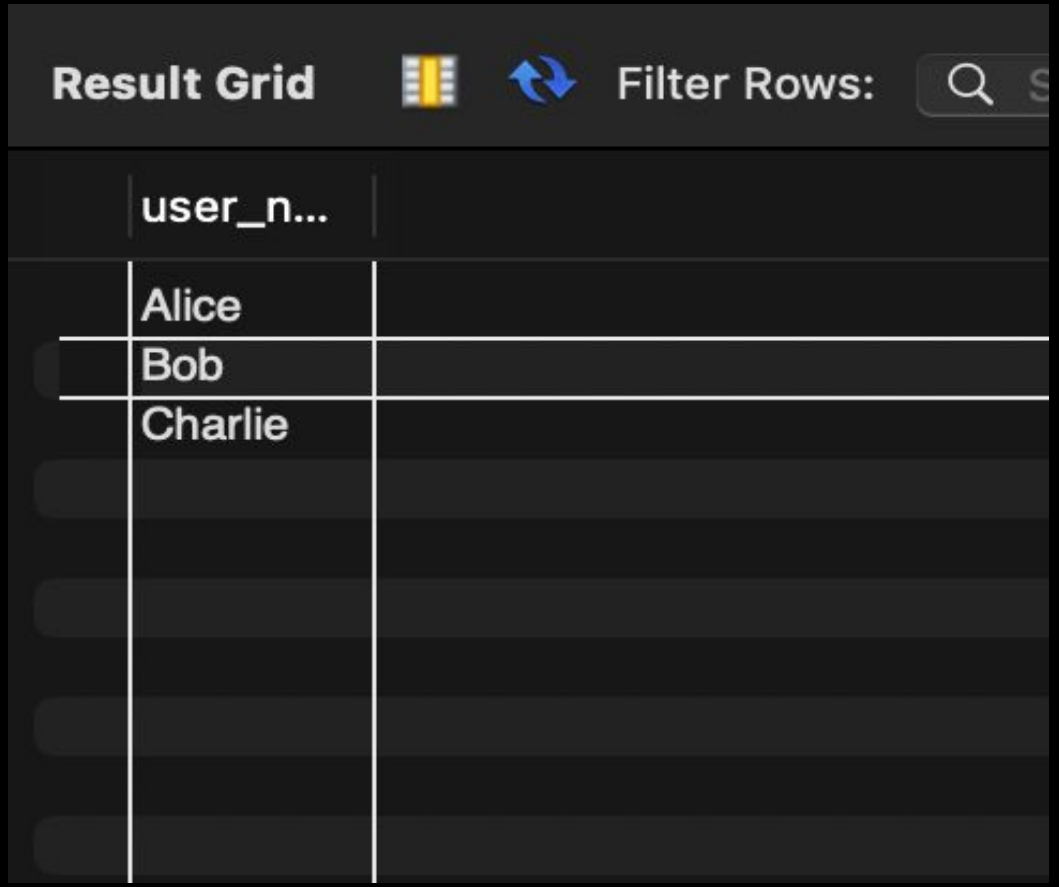
- WHERE IN Clauses
- EXISTS Logic
- Correlated Subqueries

TARGETING MARKETING FOR SPECIFIC CONTENT INTERACTIONS

Subqueries

11. WHERE clause IN: Find the names of users who watched a movie released in 2022 (Do this using a subquery, not a JOIN).

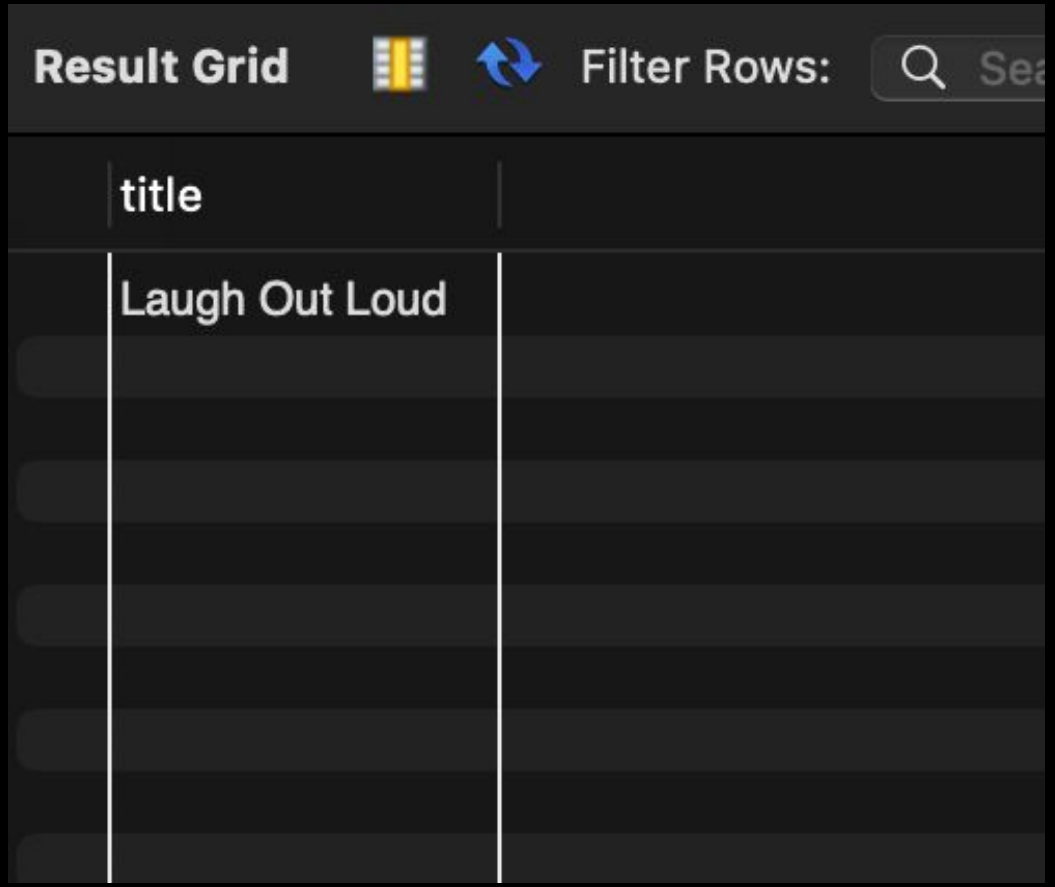
```
select user_name from users
        where user_id in (
    select user_id from watch_history
                where content_id in (
                        select content_id
            from content where release_year =2022
            )
)
;
```



Subqueries

12. Correlated Subquery: Find the title of content whose average review rating is higher than the overall average rating of all content combined.

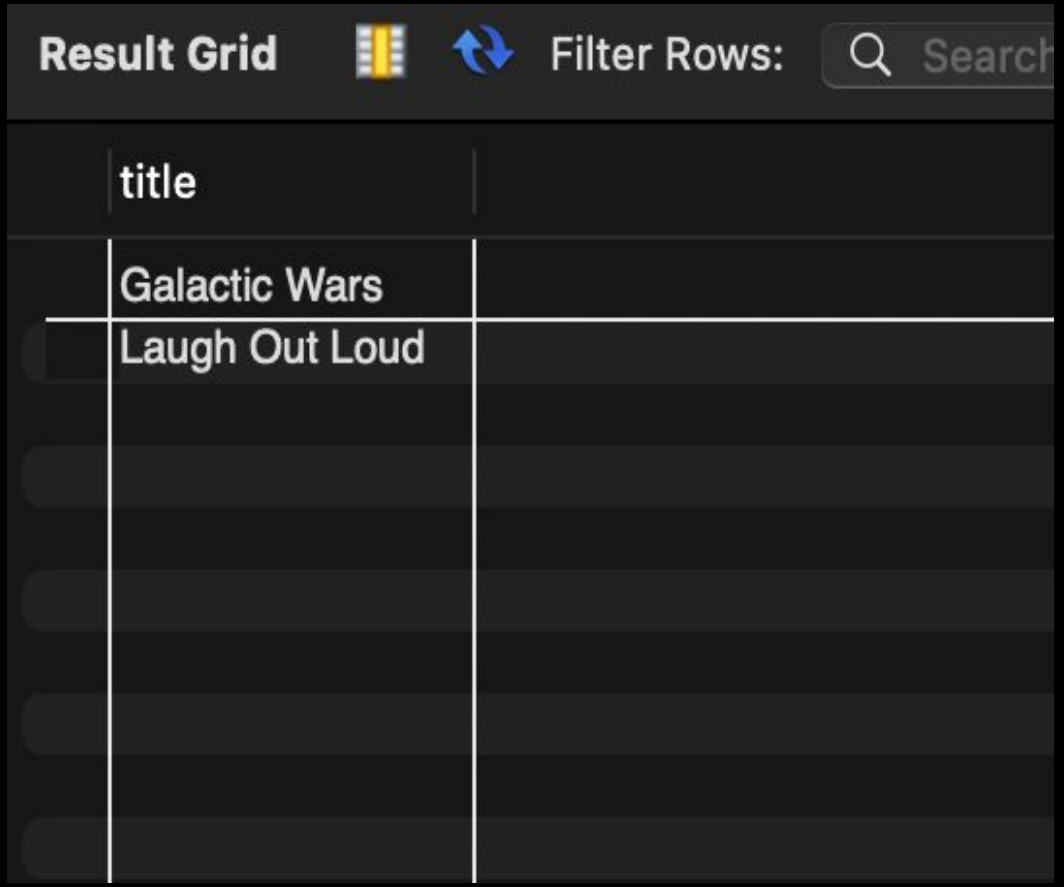
```
select c.title from content c
where (
                select avg(r.rating) from reviews r
        where r.content_id = c.content_id
        ) > (
                                select avg(r2.rating)
                        from reviews r2
                        )
;
```



Subqueries

13. EXISTS: Write a query using EXISTS to find all Content titles that have at least one review.

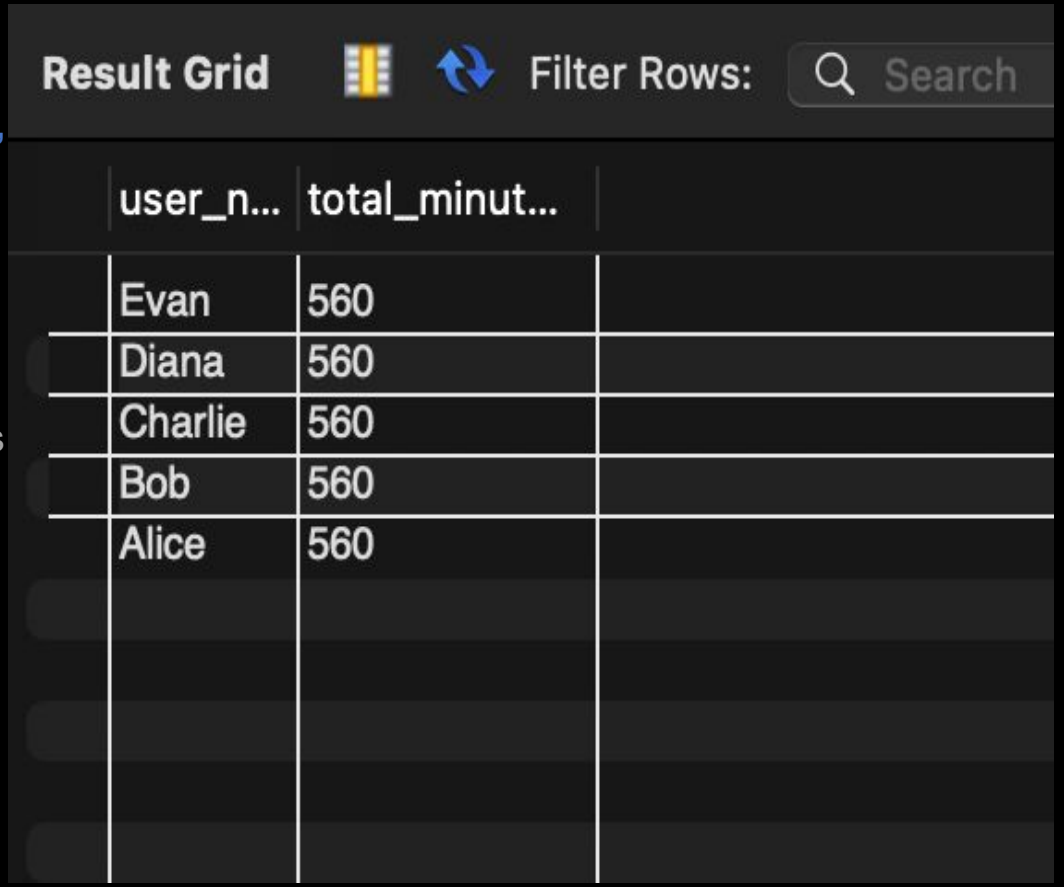
```
select c.title from content c
        where exists ( select 1 from reviews r
                                where r.content_id
= c.content_id)
;
```



Subqueries

14.FROM clause (Derived Table): Create a subquery that calculates the total watch minutes per user, and then in the outer query, filter for users who have watched more than 100 total minutes.

```
select t.user_name, t.total_minutes
        from (
                        select u.user_name,
sum(w.duration_watched_mins) as total_minutes
          from users u
          join watch_history w on u.user_id =
u.user_id
          group by u.user_id, u.user_name
          ) t
        where t.total_minutes > 100 ;
```

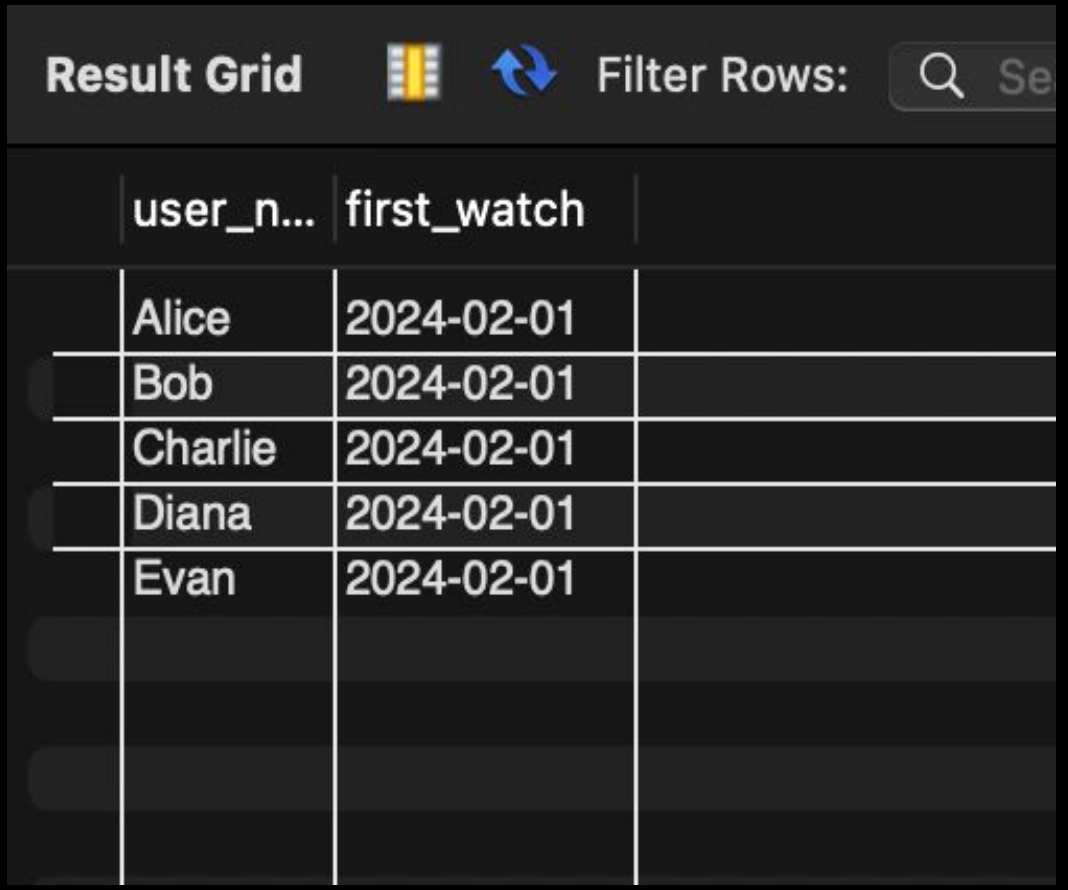


user_n...	total_minut...
Evan	560
Diana	560
Charlie	560
Bob	560
Alice	560

Subqueries

15. Scalar Subquery: Display every user_name alongside a column showing the date of the very first watch record in the entire database.

```
select u.user_name,
        ( select min(w.watch_date)
        from watch_history w) as first_watch
from users u
;
```



user_n...	first_watch
Alice	2024-02-01
Bob	2024-02-01
Charlie	2024-02-01
Diana	2024-02-01
Evan	2024-02-01

EPISODE 04

ADVANCED ANALYTICS

TECHNICAL TRANSFORMATION

CASE STATEMENTS (PIVOTING)

Aggregating user counts across Premium, Standard, and Basic tiers into a single-row executive summary.

COMMON TABLE EXPRESSIONS (CTES)

Defining "Binge Watchers" by isolating users with >100 minutes of watch time on a single date.

DATE MANIPULATION

Using DATE_ADD and HAVING to calculate the speed of first-time engagement within 30 days.

USER SEGMENTATION

Quantifying the high-value 'Premium' user base for targeted content acquisition.

RETENTION VELOCITY

Measuring how quickly new subscribers convert into active viewers.

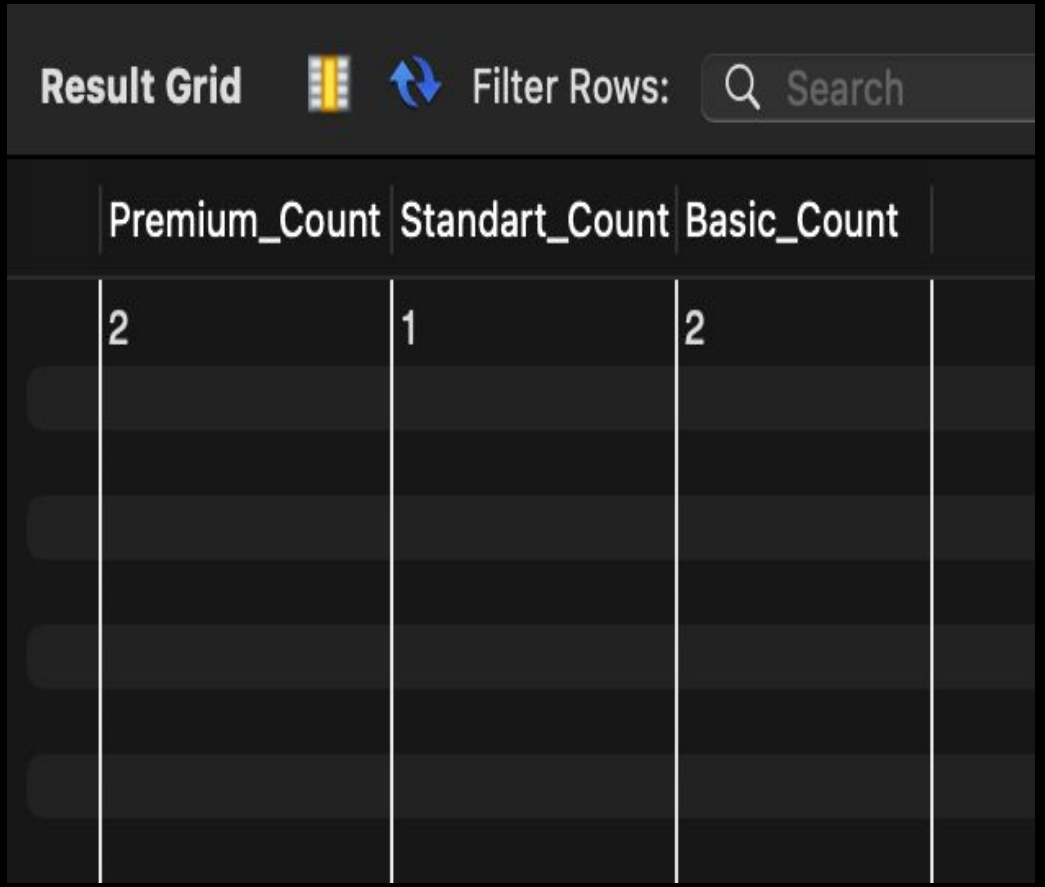
BEHAVIORAL PROFILING

Identifying extreme engagement patterns to optimize recommendation engines.

Advanced & Random Questions

16. CASE Statements (Pivot): Write a query to count how many users are in each subscription tier. Output three columns: Premium_Count, Standard_Count, and Basic_Count (Hint: Use SUM and CASE WHEN).

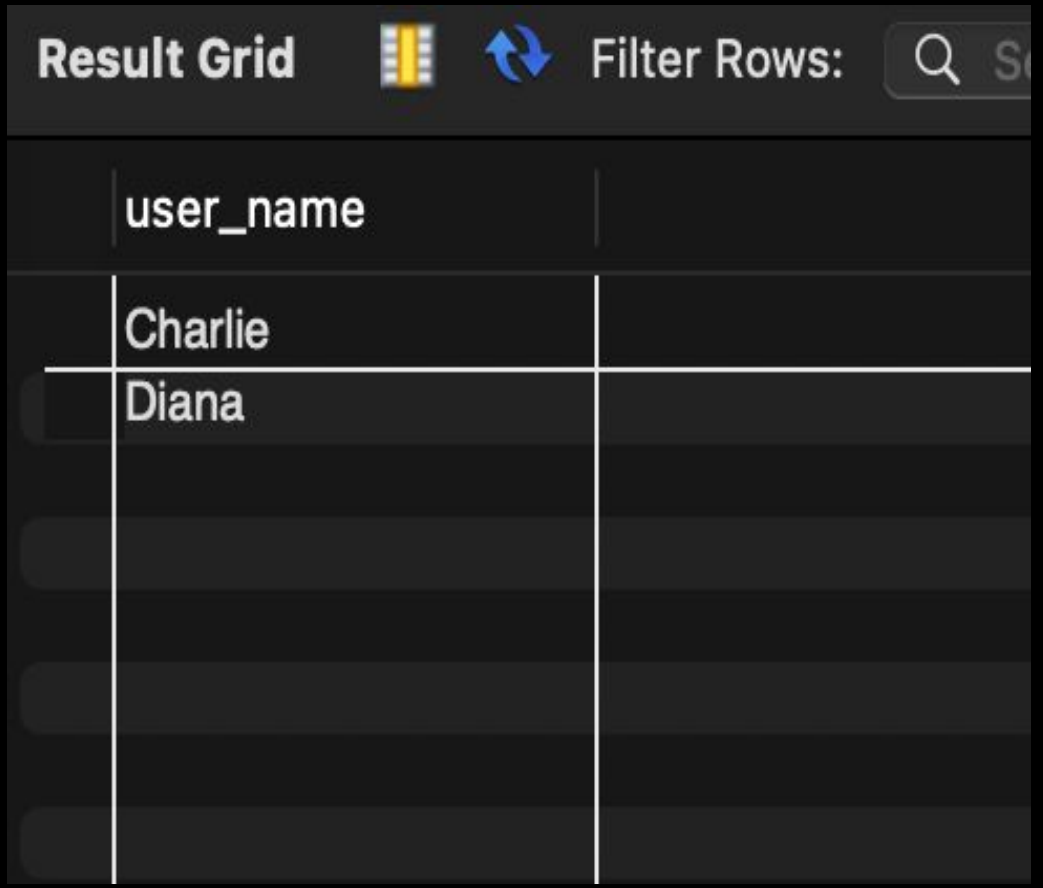
```
select
        sum(case when subscription_tier =
'Premium' then 1 else 0 end) as Premium_Count,
    sum(case when subscription_tier =
'Standard' then 1 else 0 end) as Standart_Count,
    sum(case when subscription_tier = 'Basic'
then 1 else 0 end) as Basic_Count
from users;
```



Advanced & Random Questions

17. Date Manipulation: Find users who joined the platform and watched their first piece of content within 300 days of their join_date.

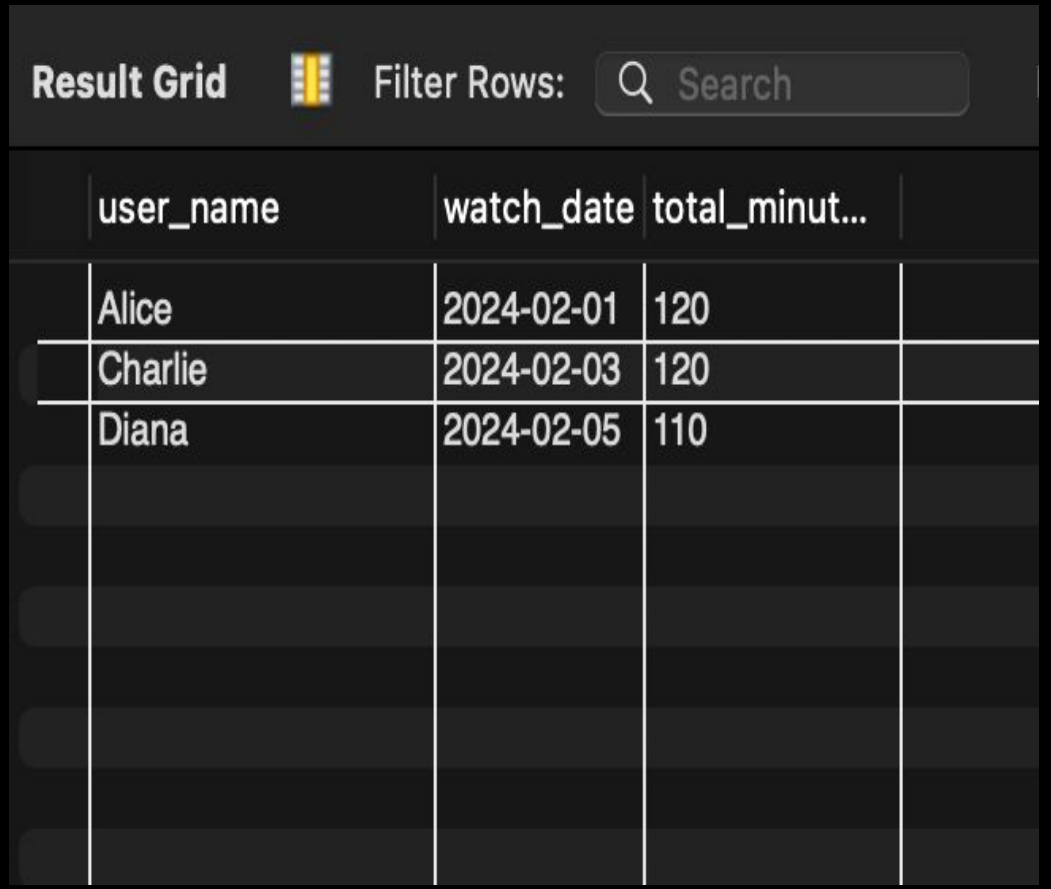
```
select u.user_name from users u
join watch_history w on u.user_id = w.user_id
group by u.user_id, u.user_name, u.join_date
having min(w.watch_date) <=
date_add(u.join_date, interval 300 day)
;
```



Advanced & Random Questions

18. Common Table Expressions (CTEs): Use a CTE to find all "Binge Watchers" — defined as users who have watched more than 100 minutes of content on a single watch_date.

```
WITH daily_watch AS (
    SELECT user_id, watch_date,
        SUM(duration_watched_mins) AS
total_minutes
FROM watch_history
GROUP BY user_id, watch_date
)
SELECT u.user_name, d.watch_date,
d.total_minutes
FROM daily_watch d
JOIN users u ON u.user_id = d.user_id
WHERE d.total_minutes > 100;
```

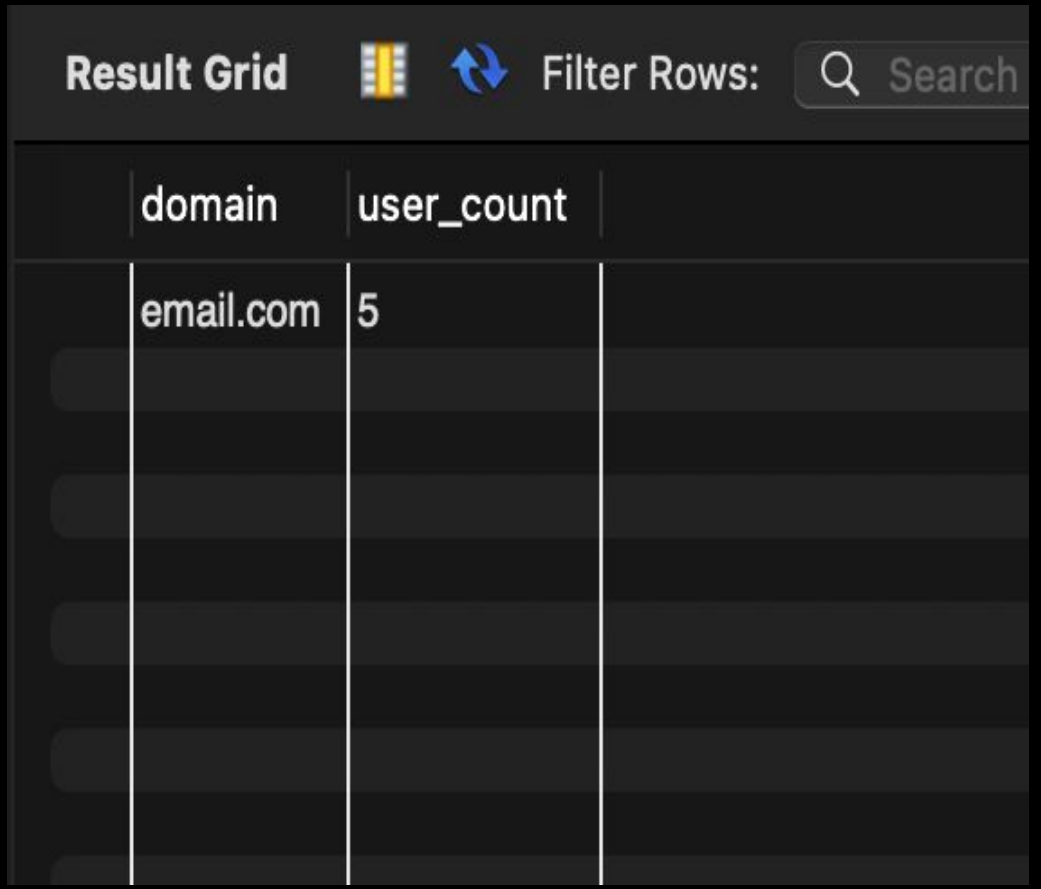


user_name	watch_date	total_minut...
Alice	2024-02-01	120
Charlie	2024-02-03	120
Diana	2024-02-05	110

Advanced & Random Questions

19. String Functions: Extract the email domain (everything after the @ symbol) for each user and count how many users belong to each domain.

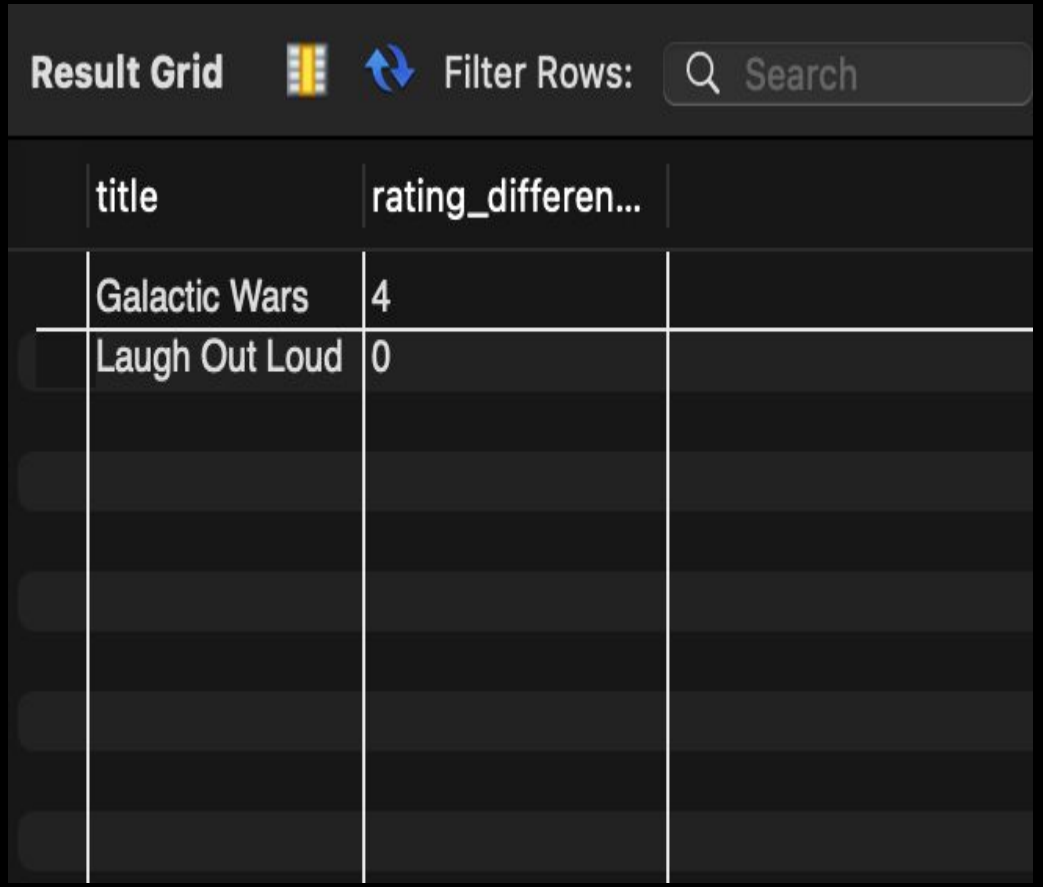
```
SELECT
    SUBSTRING_INDEX(email, '@', -1) AS
domain,
    COUNT(*) AS user_count
FROM users
GROUP BY domain;
```



Advanced & Random Questions

20. The Grand Finale (Complex Logic): Find the "Most Polarizing" content. This is the content that has the biggest difference between its highest review rating and its lowest review rating. Display the title and the rating_difference.

```
SELECT
    c.title,MAX(r.rating) - MIN(r.rating) AS
rating_difference
FROM content c
JOIN reviews r
    ON c.content_id = r.content_id
GROUP BY c.content_id, c.title
ORDER BY rating_difference DESC
;
```



title	rating_differen...
Galactic Wars	4
Laugh Out Loud	0

TOP 10 DATA DISCOVERIES

1. **HIGHEST WATCH GENRE**
   Action & Drama dominance in engagement.
2. **PREMIUM PREFERENCES**
   Content choices of high-value users.
3. **BINGE WATCHERS**
   Users with >100 mins/day activity.
4. **RATING TRENDS**
   Content ranking by user satisfaction.
5. **TIER DISTRIBUTION**
   Count of Premium vs. Basic users.
6. **RETENTION SPEED**
   Time from join to first watch record.
7. **DOMAIN ANALYSIS**
   Email provider distribution across users.
8. **CONTENT QUARTILES**
   Release year distribution of catalog.
9. **POLARIZING CONTENT**
   High variance in user review scores.
10. **FIRST WATCH RECORDS**
    Database-wide engagement start dates.

THE FEATURE PRESENTATION

THE GRAND FINALE

POLARIZING CONTENT

We identified the "Most Polarizing" content by calculating the maximum variance between user ratings. This metric reveals content that triggers strong, opposing reactions—crucial for niche targeting.

"LOVE IT OR HATE IT" ANALYSIS

COMPLEX LOGIC

```
SELECT c.title,
MAX(r.rating) - MIN(r.rating)
AS rating_difference
FROM content c
JOIN reviews r ON c.id = r.id
GROUP BY c.title
ORDER BY rating_difference DESC;
```

This analysis helps StreamFlix refine recommendation algorithms by distinguishing between universally liked content and high-engagement niche titles.

CURATED FOR YOU

MY LIST: SQL TOOLKIT

SCHEMA DESIGN

Architecting relational structures with primary/foreign key integrity.

COMPLEX JOINS

Multi-table Inner, Left, and Right joins for deep data integration.

WINDOW FUNCTIONS

Advanced ranking, running totals, and sequential data analysis.

SUBQUERIES

Nested logic, correlated subqueries, and EXISTS filtering.

CTES

Common Table Expressions for modular and readable query logic.

PIVOTING & AGGREGATION

Transforming raw records into high-level business metrics.

TECHNICAL COMPETENCIES APPLIED IN STREAMFLIX PROJECT

CREDITS & METHODOLOGY

LEAD ANALYST

Imtiyaj Ali Shaikh

PRODUCTION

SQL Database Management Project

DATA SOURCE

StreamFlix Database

TECHNICAL APPROACH

Relational Database Schema Design & Normalization

Complex Multi-table Joins for User-Content Mapping

Advanced Window Functions for Behavioral Trends

Subquery Optimization for Precision Filtering

BI Analysis for Subscriber Retention Metrics

THANKS

THE END